

1. What is Static Analysis?

Answer:

Static analysis is the process of analyzing source code without executing it, mainly to detect bugs, enforce coding standards, and improve code quality early.

2. What is the difference between Static Checking and Dynamic Checking?

Answer:

Static checking finds errors before execution (compile time), while dynamic checking finds errors during execution (runtime).

3. Explain Static Typing in Java.

Answer:

Static typing means that variable types are known at compile time, allowing the compiler to detect type-related errors early.

4. Give examples of errors caught by Static Checking.

Answer:

Syntax errors, wrong method names, wrong number of arguments, type mismatches, and incorrect return types.

5. Give examples of errors caught by Dynamic Checking.

Answer:

Divide-by-zero errors, array index out of bounds, and errors caused by specific runtime values.

6. What are the benefits of Static Analysis?

Answer:

Early bug detection, improved security, enforcement of coding standards, and measuring code complexity.

7. What is the Hailstone Sequence?

Answer:

A sequence where if n is even, $n = n/2$, and if n is odd, $n = 3n + 1$, repeating until n reaches 1.

8. Why is Java considered a statically typed language?

Answer:

Because all variable types are checked and determined by the compiler before the program runs.

9. What is the difference between primitive types and object types in Java?

Answer:

Primitive types store actual values, while object types store references (memory addresses) to objects.

10. Why are Java primitive numeric types not true mathematical numbers?

Answer:

Because of integer division, integer overflow, and special floating-point values like NaN and Infinity.

11. Explain integer overflow in Java.

Answer:

When a calculation exceeds the maximum or minimum value of a type, the result wraps around silently without error.

12. What is Static Analysis able to detect in code?

Answer:

Type mismatches, uninitialized variables, dead code, resource leaks, and coding standard violations.

13. What is FindBugs?

Answer:

A static analysis tool that examines Java bytecode to detect bug patterns and classify defects by severity.

14. What are the main categories of bugs in FindBugs?

Answer:

Correctness, Bad Practice, Performance, Multithreaded Correctness, Security, and Malicious Code.

15. What is Checkstyle?

Answer:

A static analysis tool that focuses on enforcing Java coding style and formatting rules.

16. What is the difference between Arrays and Lists in Java?

Answer:

Arrays have fixed length, while Lists are variable-length and can grow dynamically.

17. Why can array overflow bugs be dangerous?

Answer:

Because they can cause runtime crashes and security vulnerabilities if not properly checked.

18. What is a mutable type?

Answer:

A type whose object values can be changed after creation.

19. Give examples of immutable types in Java.

Answer:

String, primitive types, wrapper classes, BigInteger, and BigDecimal.

20. What does mutating a value mean?

Answer:

Changing the internal contents of a mutable object, such as modifying a list or array.

21. What is an immutable reference?

Answer:

A reference declared with final that cannot be reassigned after initialization.

22. Can a final reference point to a mutable object?

Answer:

Yes, the reference cannot change, but the object it points to can still be mutated.

23. What is aliasing?

Answer:

Aliasing occurs when multiple variables reference the same mutable object.

24. Why is aliasing dangerous with mutable objects?

Answer:

Because changes through one reference affect all other references unexpectedly.

25. Why are immutable types safer than mutable types?

Answer:

They prevent unintended side effects, make programs easier to understand, and reduce bugs.

26. What problem occurs when modifying a list during iteration?

Answer:

A ConcurrentModificationException is thrown at runtime.

27. What is a snapshot diagram?

Answer:

A diagram that visualizes variables and the values or objects they reference at runtime.

28. Why is returning mutable objects risky?

Answer:

Because callers can modify the returned object and unintentionally change internal program state.

29. How can risks of mutation be reduced?

Answer:

By using immutable objects or returning copies of mutable objects.

30. Why is documenting assumptions important in programming?

Answer:

It improves readability, prevents misunderstandings, and helps maintain and modify code safely.